LANGGRAPH

Core Concepts & Agentic AI

Source: CampusX — Agentic AI using LangGraph (Session 4)

Framework	LangGraph (by LangChain)
Prerequisites	Python, LangChain basics, LLM fundamentals
Key Library	langgraph (pip install langgraph)
Difficulty	Intermediate

These notes include original lecture content PLUS additional diagrams, code, tables, and learning aids.

TABLE OF CONTENTS

1. What is LangGraph? — Introduction & Motivation
2. Five Common LLM Workflow Patterns
   - 2A. Prompt Chaining
   - 2B. Routing
   - 2C. Parallelization
   - 2D. Orchestrator-Worker
   - 2E. Evaluator-Optimizer (The Agentic Loop)
3. The Graph Structure — Nodes & Edges
4. State — The Memory of Your Graph
5. Reducers — Handling State Updates
6. Execution Model — Under the Hood (Pregel)
7. Extra: LangGraph vs LangChain Comparison
8. Extra: Quick-Start Code Template
9. Summary & Self-Assessment Checklist

1. What is LangGraph?

LangGraph is an **orchestration framework** designed to build stateful, multi-actor applications with Large Language Models (LLMs). It is developed by the same team behind LangChain but serves a fundamentally different purpose.

Core Idea

LangGraph takes a linear or complex LLM workflow and converts it into a graph structure where:

- **Nodes** represent discrete tasks — Python functions, API calls, or LLM invocations.
- **Edges** represent the control flow — the rules for moving from one task to another.

Why Not Just Use LangChain?

LangChain excels at building linear chains (A → B → C). However, real-world AI agents need:

Capability	LangChain	LangGraph
Linear Chains	✓ Yes	✓ Yes
Loops / Retries	✗ No	✓ Yes (Cyclic Graphs)
Conditional Branching	Limited	✓ Full Support
Parallel Execution	✗ No	✓ Yes (Supersteps)
Persistent Memory	✗ No	✓ Built-in State
Human-in-the-Loop	✗ No	✓ Yes

> ✨ **Key Insight**
> LangGraph models workflows as State Machines / Directed Graphs. This means your AI agent can think, act, evaluate, and retry — just like a human problem-solver.

Analogy: Chef in a Kitchen

LangChain = Following a recipe step by step (chop → cook → plate). **LangGraph** = A head chef managing a kitchen — can assign tasks to multiple sous-chefs in parallel, taste-test and send food back to be re-cooked (loops), and route orders to different stations based on the dish type (branching).

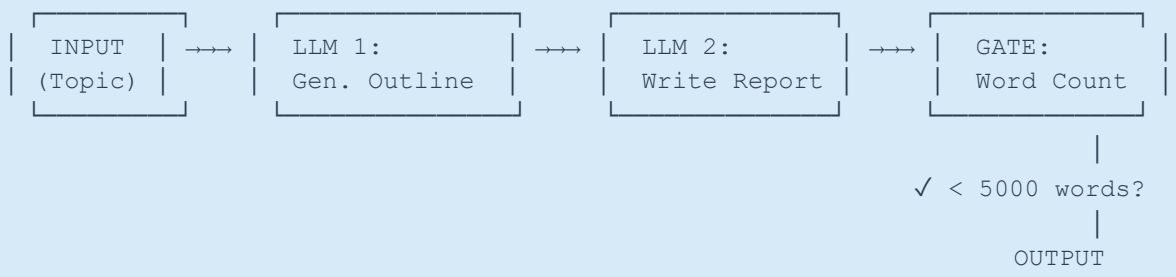
2. Five Common LLM Workflow Patterns

These five architectural patterns form the building blocks of Agentic AI. They range from simple sequential pipelines to complex self-correcting loops.

2A. Prompt Chaining

Concept: A linear sequence where the output of one LLM call serves as the input for the next. Used when a single task is too complex for one prompt.

Flow Diagram: Prompt Chaining



Example: Creating a report — User provides a topic → LLM 1 generates an outline → LLM 2 writes the full report → Gate checks if word count < 5000.

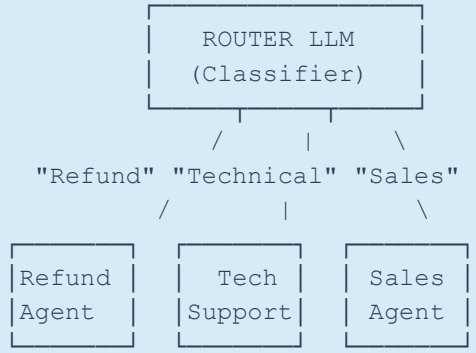
➕ Extra: When to Use Prompt Chaining

Use when a single prompt would be too long/complex, when you need intermediate validation (gates), or when different prompts need different system instructions (e.g., outliner vs. writer).

2B. Routing

Concept: Using an LLM as a classifier/router to decide the downstream path. It introduces conditional logic (If/Else) into the workflow.

Flow Diagram: Routing



Example: Customer Support Chatbot — The LLM analyzes the user query and routes it to the specific department (Refunds vs. Tech Support vs. Sales).

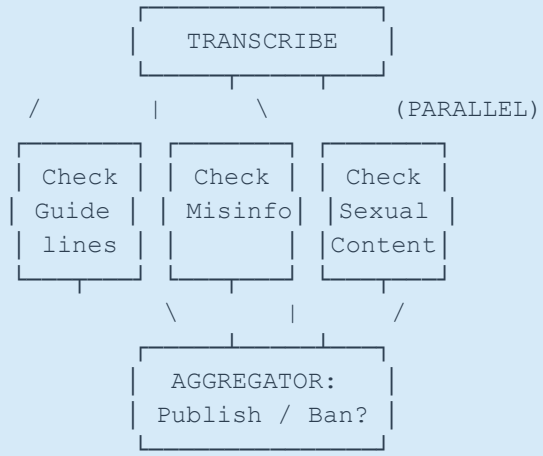
✚ Extra: Routing Implementation

In LangGraph, routing is done via Conditional Edges. The router function returns a string (e.g., 'refund', 'tech', 'sales') and the graph maps each string to the corresponding node.

2C. Parallelization

Concept: Breaking a task into independent sub-tasks that run simultaneously, then aggregating results. This significantly reduces latency.

Flow Diagram: Parallelization (YouTube Moderation Example)



Example: YouTube Content Moderation — A video is checked against three criteria simultaneously. The final decision depends on combined results.

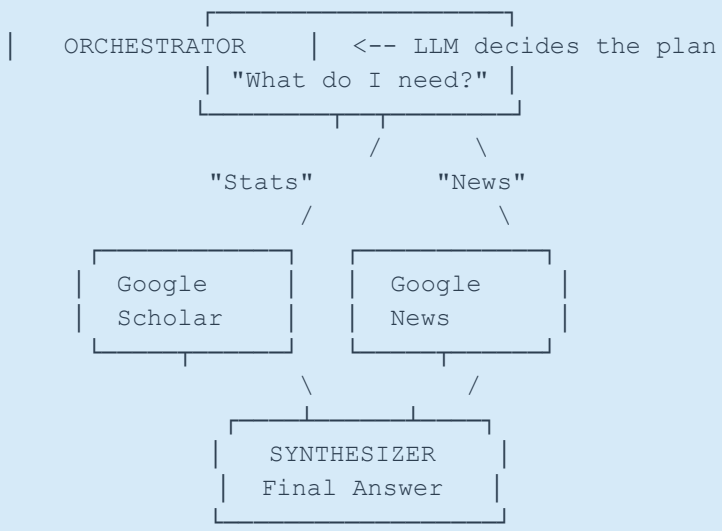
Parallelization vs. Orchestrator-Worker

Aspect	Parallelization	Orchestrator-Worker
Sub-tasks	Known in advance (static)	Decided at runtime (dynamic)
Control	Deterministic	LLM-driven
Flexibility	Fixed branches	Adaptive branches
Example	Content moderation checks	Research assistant

2D. Orchestrator-Worker

Concept: Similar to parallelization but dynamic. The "Orchestrator" LLM first creates a plan (determines what sub-tasks are needed) and then assigns them to workers. Sub-tasks are NOT known in advance.

Flow Diagram: Orchestrator-Worker

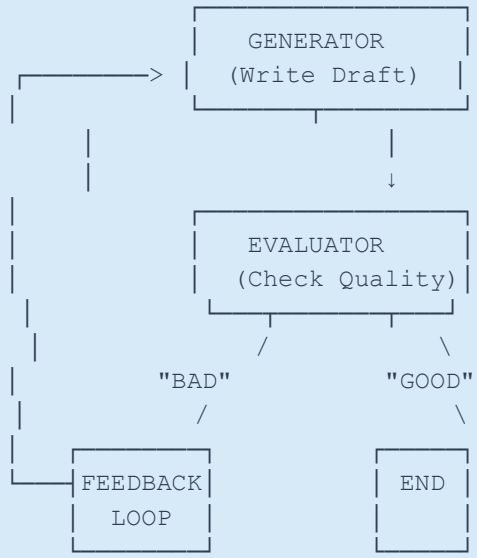


Example: Research Assistant — Query "Quantum Physics" triggers Google Scholar worker; query "Election Results" triggers Google News worker. The orchestrator decides dynamically.

2E. Evaluator-Optimizer (The Agentic Loop)

Concept: This is the most "agentic" pattern. A self-correcting loop where a Generator produces an attempt, an Evaluator critiques it, and the cycle repeats until quality is met.

Flow Diagram: Evaluator-Optimizer Loop



Example: Writing a blog post — LLM 1 drafts it, LLM 2 plays the "critic." They loop until quality standards are met.

> ⚠ **Warning: Infinite Loops**
> Always set a max_retries or iteration limit in Evaluator-Optimizer patterns. Without it, a harsh critic + weak generator = infinite loop. A common limit is 3–5 iterations.

Summary: All 5 Patterns at a Glance

Pattern	Structure	Key Feature	Best For
Prompt Chaining	Linear (A→B→C)	Sequential + Gates	Multi-step content generation
Routing	Branching (1→N)	Conditional Edges	Classification & dispatch
Parallelization	Fan-out + Fan-in	Simultaneous execution	Multi-criteria evaluation
Orchestrator-Worker	Dynamic Fan-out	LLM plans sub-tasks	Research & complex queries
Evaluator-Optimizer	Cyclic Loop	Self-correction	Quality-critical generation

3. The Graph Structure — Deep Dive

Case Study: UPSC Essay Writing Platform — A student writes and revises an essay with AI feedback.

3.1 Nodes (The 'What')

Each node is a standard Python function. It receives the current State, performs computation, and returns updates to the State.

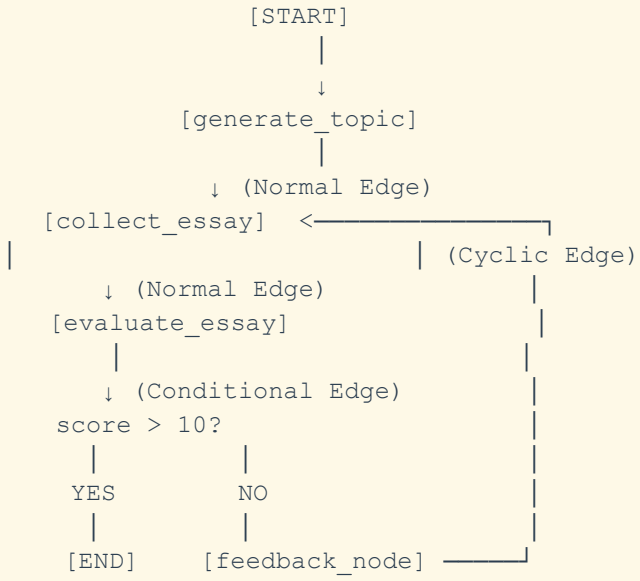
Node Name	Function	Description
generate_topic()	Topic Selection	Presents a random UPSC essay topic
collect_essay()	User Input	Collects the student's essay text
evaluate_essay()	AI Grading	LLM evaluates essay on multiple criteria
feedback_node()	Provide Feedback	Gives structured feedback to student

3.2 Edges (The 'When')

Edges connect nodes to define control flow. LangGraph supports three types:

Edge Type	Description	Example
Normal Edge	Always moves A → B	generate_topic → collect_essay
Conditional Edge	Branch logic (if/else)	If score > 10 → END; else → feedback
Cyclic Edge	Points back to previous node	feedback → collect_essay (rewrite loop)

UPSC Essay Platform — Full Graph



✨ Key Insight: Cyclic vs. Acyclic Graphs

LangChain = DAG (Directed Acyclic Graph — no loops). LangGraph = Directed Graph that CAN have cycles. This is what makes agents possible — they can retry, reflect, and improve.

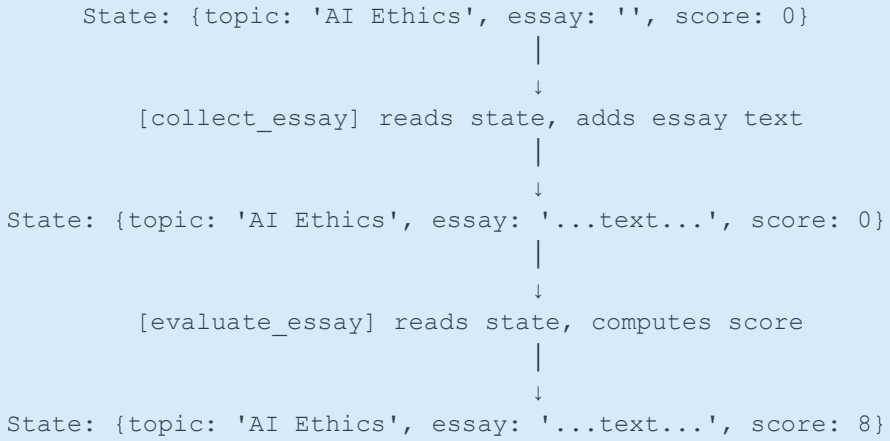
4. State — The Memory of Your Graph

This is arguably the most critical concept in LangGraph. In standard Python, variables die when functions end. In LangGraph, State persists across the entire workflow.

What is State?

A shared dictionary (**TypedDict**) that is passed to every node. Nodes can read from the state AND write updates to it.

How State Flows Through Nodes



Code: Defining State

```
from typing import TypedDict


class EssayState(TypedDict):
    topic: str           # The essay topic
    essay_text: str      # The student's essay
    scores: dict         # Evaluation scores
    retry_count: int     # How many attempts
```

+ Extra: Why TypedDict?

TypedDict gives you type safety and IDE autocompletion. It tells LangGraph the shape of your state, so every node knows exactly what keys are available.

Analogy: State = Shared Clipboard

Imagine a team of workers sharing a single clipboard. Each worker picks up the clipboard, reads the information, does their work, writes their results back, and passes it to the next worker. The clipboard is the State.

5. Reducers — Handling State Updates

The Problem

By default, when a node returns a value for a state key, it **OVERWRITES** the existing value.

Scenario: In a chatbot, if User says "Hi" (msg 1) and then "My name is Nitish" (msg 2), overwriting would delete "Hi". The bot loses all context.

The Solution: Reducers

A reducer is a function that defines HOW to update a specific key in the state.

Behavior	What Happens	Analogy
Default (Overwrite)	New value replaces old value	Whiteboard: erase old, write new
Reducer (Append)	New value is added to a list	Notebook: write below the last entry

Visual: Overwrite vs. Append

```
WITHOUT REDUCER (Overwrite):
  Step 1: messages = ["Hi"]
Step 2: messages = ["My name is Nitish"]  ← "Hi" is LOST!

WITH REDUCER (Append):
  Step 1: messages = ["Hi"]
Step 2: messages = ["Hi", "My name is Nitish"]  ← History PRESERVED!
```

Code: Using Reducers in LangGraph

```
from langgraph.graph.message import add_messages
from typing import Annotated, TypedDict

class ChatState(TypedDict):
    # Annotated tells LangGraph to use add_messages as the reducer
    messages: Annotated[list, add_messages]
```

How it works: The **Annotated[list, add_messages]** syntax tells LangGraph: 'When a node returns new messages, APPEND them to the list instead of replacing it.'

Built-in Reducers in LangGraph

Reducer	Behavior	Use Case
add_messages	Appends new messages to list	Chat history
operator.add	Concatenates lists	Accumulating results
Custom function	Any custom merge logic	Deduplication, scoring, etc.

✨ Key Insight

Reducers are what make LangGraph chatbots 'remember' conversation history. Without them, every new message would wipe the slate clean.

6. Execution Model — Under the Hood

LangGraph's engine is inspired by Google Pregel, a system designed for large-scale graph processing.

Step 1: Compile

The user defines the graph (nodes + edges). LangGraph compiles it into a runnable executable, checking for errors like orphaned nodes or invalid edges.

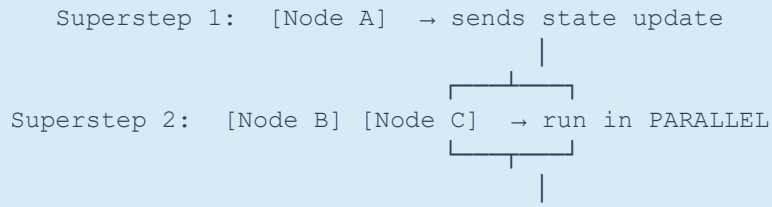
Step 2: Invocation

The graph starts executing with an initial state passed by the user.

Step 3: Supersteps

LangGraph executes in discrete rounds called **Supersteps**. If Node A branches into Node B and Node C (parallel), both B and C run in the SAME Superstep.

Superstep Execution Timeline



Step 4: Message Passing

Nodes do NOT call each other directly. Node A finishes and updates the State. The Graph engine reads the state update and triggers the next appropriate node(s). This is similar to event-driven architecture.

Traditional Approach	LangGraph Approach
function_A() calls function_B()	Node A updates state; engine triggers Node B
Tight coupling between functions	Loose coupling via shared state
Hard to add parallelism	Parallelism is built-in
No automatic retries	Cyclic edges enable retries

7. Extra: LangGraph vs. LangChain — When to Use What

This section is additional context not from the video, provided to clarify a common source of confusion.

Dimension	LangChain	LangGraph
Purpose	Building LLM chains & RAG pipelines	Building stateful AI agents
Topology	Linear DAGs only	Cyclic + Acyclic graphs
State	Passed via chain; no persistence	First-class, persistent state
Control Flow	Sequential	Branches, loops, parallel
Memory	Conversation buffer (manual)	Built-in with reducers
Human-in-Loop	Not supported natively	Built-in breakpoints
Complexity	Simpler, faster to build	More powerful, more setup
Best For	RAG, Q&A, summarization	Agents, multi-step workflows

✨ Rule of Thumb

If your workflow has NO loops and NO conditional branching, use LangChain. The moment you need retry logic, self-correction, or dynamic routing, switch to LangGraph.

8. Extra: Quick-Start Code Template

A minimal but complete LangGraph example to get you started. This implements a simple 2-node graph.

```
# pip install langgraph langchain-openai

from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

# 1. DEFINE STATE
class MyState(TypedDict):
    messages: Annotated[list, add_messages]
    current_step: str

# 2. DEFINE NODES (plain Python functions)
def greet(state: MyState) -> dict:
    return {"messages": ["Hello! How can I help?"],
            "current_step": "greeted"}

def respond(state: MyState) -> dict:
    user_msg = state["messages"][-1]
    return {"messages": [f"You said: {user_msg}"],
            "current_step": "responded"}

# 3. BUILD GRAPH
graph = StateGraph(MyState)
graph.add_node("greet", greet)
graph.add_node("respond", respond)

# 4. ADD EDGES
graph.add_edge(START, "greet")
graph.add_edge("greet", "respond")
graph.add_edge("respond", END)

# 5. COMPILE & RUN
app = graph.compile()
result = app.invoke({"messages": ["Hi there!"],
                     "current_step": ""})
```

```
print(result)
```

➕ Extra: Adding a Conditional Edge

To add routing, use graph.add_conditional_edges("node_name", routing_function, {"route1": "node_a", "route2": "node_b"}). The routing function returns a string key that maps to the target node.

9. Summary & Self-Assessment Checklist

Key Takeaways

1. LangGraph = Graph-based orchestration for stateful LLM agents (built on top of LangChain ecosystem).

2. Five patterns: Prompt Chaining, Routing, Parallelization, Orchestrator-Worker, Evaluator-Optimizer.

3. Nodes = Python functions (the logic). Edges = control flow (the path).

4. State = shared TypedDict passed to every node. It IS the memory.

5. Reducers prevent state overwriting — essential for chat history (use add_messages).

6. Execution uses Supersteps (parallel rounds) and Message Passing (loose coupling).

Self-Assessment Checklist

Test yourself — can you answer each of these?

- ☐ What is the difference between a DAG (LangChain) and a cyclic graph (LangGraph)?
- ☐ Draw the Evaluator-Optimizer loop from memory.
- ☐ Why do we use TypedDict for State instead of a regular dictionary?
- ☐ Explain WHY a chatbot needs a reducer for its message history.
- ☐ Which component contains the actual logic — Nodes or Edges?
- ☐ What is a Superstep and why is it different from a regular step?
- ☐ When would you choose Orchestrator-Worker over Parallelization?
- ☐ What happens if you forget to set a max_retries in an Evaluator-Optimizer loop?

What's Next?

In the next session, these concepts will be applied hands-on to build a real LangGraph agent. Practice the code template above and make sure you can explain each of the 5 patterns.